

CS2610: Computer Organization and Architecture

Lab

Introductory Lab: Hello RISC-V !!!

23rd January 2026

Introduction

Welcome to the first lab on RISC-V Assembly Programming! In this lab, we will explore the basics of RISC-V assembly using the Spike simulator. This hands-on exercise is designed to give you a foundational understanding of how programs written in C and assembly interact with the underlying hardware at the register level.

In this lab, you will:

1. Simulate an assembly program and analyze the state of the registers after executing each instruction.
2. Compile and run a simple C program to observe the changes in the registers during execution.

By the end of this lab, you will have a better understanding of how instructions are executed and how data is manipulated at the hardware level in the RISC-V architecture.

1 Part 1: Simulating an Assembly Program

Analyze the provided RISC-V assembly code (`program.S`):

```
.section .data # Data section
num1: .word 15 # First number (15)
num2: .word 10 # Second number (10)
result: .word 0 # Variable to store the result

.section .text # Code section
.globl main # Entry point for the program

main:
    # Load the numbers into registers
    la t0, num1 # Load address of num1 into t0
    lw t1, 0(t0) # Load the value at address t0 (num1) into t1
    la t0, num2 # Load address of num2 into t0
    lw t2, 0(t0) # Load the value at address t0 (num2) into t2

    # Perform the addition
    mul t3, t1, t2 # Multiply the values in t1 and t2, store the result in t3

    # Store the result back into memory
    la t0, result # Load address of result into t0
    sw t3, 0(t0) # Store the value in t3 (result) into memory

    # Exit the program
    li a7, 93 # Load ecall number for exit into a7
    ecall # Make the system call
```

Compilation

Compile the RISC-V assembly program:

```
$ riscv64-unknown-elf-gcc -o my_RISCV_executable program.S
```

Simulation

Run the program in Spike debug mode:

```
$ spike -d pk my_RISCV_executable
```

Execute the program instruction-by-instruction and observe:

- Instruction executed
- Contents of all registers
- Changes in the program counter (PC)

Questions

- What is the address of `main` function?
- How does each instruction affect the register states?
- What are the addresses of the data variables `num1`, `num2`, `result`
- Create a dump file. Are there differences between the asm code and the dump? Explain them.

2 Part 2: Observing Register States with C Code

Write a simple C program:

```
int mul(int a, int b)
{
    int c = 0;
    c = a * b;
    return c;
}

int main()
{
    int x = 20;
    int y = 10;
    int z;
    z = mul(x, y);
    return z;
}
```

Compilation

```
$ riscv64-unknown-elf-gcc -o my_RISCV_executable my_C_code.c
```

Execution

```
$ spike pk my_RISCV_executable
$ spike -d pk my_RISCV_executable
```

Questions

- Which registers store intermediate results?
- Which register contains the final result at the end of `main()`?
- Compare and explain differences between the generated ASM code and original ASM code from part 1

3 Useful References

- RISC-V ISA Manual: Volume I (Unprivileged Architecture)
- RISC-V ASM programming manual
- Lecture slides and TA handouts

4 Useful Commands

```
# Compile C or assembly
$ riscv64-unknown-elf-gcc my_C_code.c
$ riscv64-unknown-elf-gcc [-nostartfiles -T linker.ld] my_riscv_asm_code.S

# Create a dump file
$ riscv64-unknown-elf-objdump -D a.out > my_dump_file

# Simulate
$ spike pk a.out
$ spike -d pk a.out

# Set paths if tools are missing
$ export PATH="/scratch/os-iitm/riscv-isa-sim":$PATH
$ export PATH="/scratch/os-iitm/install/riscv/bin":$PATH
```

Need Help?

- Spike debugging documentation
- Ask your TAs